



Mini Simulation Project (MSP) Documentation

Group Name:	AMM	Date Submitted:	Nov 28, 2025
Project Title:	AUTOMATIC INK REFILLER		
Leader:	Ephraim Joshua M. Flores	Task:	☐ Docu ☐ HW ☐ SW
Members:	Riley Aaron O. Sy	Task:	☐ Docu ☐ HW ☐ SW
	Alexa Janine Gomez	Task:	☐ Docu ☐ HW ☐ SW

I. Rationale

The increasing demand for convenience in routine tasks has driven the advancement of automated systems, especially in environments where repetitive processes consume time and effort. Manual refilling of whiteboard markers is one such task, is often messy, inconsistent, and prone to spills that may stain surfaces or clothing. Automation presents a practical solution by ensuring precision, cleanliness, and efficiency in distributing ink.

This study investigates the development of an 8086 microprocessor-based automated marker-refilling machine. Through the implementation of the microprocessor, the system enables the simultaneous refilling of up to four markers with a single command. Each slot is equipped with an automatic detection mechanism that identifies whether a marker is inserted. Only the occupied slots are activated, allowing ink to be dispensed efficiently, consistently, and without unnecessary waste. Compared to the traditional, tedious, and stain-prone method of manually loading ink into each marker, the automated approach drastically reduces the workload and eliminates the risk of accidental spills.

The prototype emphasizes the importance of controlled ink dispensing and consistent measurement. Its algorithm manages slot detection, ink delivery timing, and dispensing volume, ensuring uniform refilling across all markers. This automation is highly beneficial in school environments where markers are frequently used and need to be refilled in large batches, restoring convenience and productivity for both staff and students.

This paper will discuss the system overview, hardware design, software design, and algorithm of the automated marker refiller. The findings of this study aim to contribute to research in microprocessor-based automation and demonstrate the feasibility of using the 8086 microprocessor to streamline repetitive tasks and enhance operational efficiency.

II. System Overview

This section will detail the development of an Automatic Marker Refiller system designed to autonomously detect and replenish the ink levels of compatible whiteboard markers. The system is built around the Intel 8086 microprocessor and utilizes external peripheral control for sensing, actuation, and user feedback.

Feature	Specification	Component(s)
Microprocessor	Intel 8086	CPU
Input/Output (I/O) Control	8255 Programmable Peripheral Interface (PPI)	PORT1 (ADC/LCD), PORT2 (Switches/Pumps)
Ink Level Sensing	Analog-to-Digital Converter (ADC0808)	Conductive sensor and ADC
Refilling Mechanism	Four independent DC pumps	Controlled via 8255 Port 2A
User Interface	LM044L (4×40) Character LCD	Display system status and messages
Marker Detection	Four Slot Switches (SW1–SW4)	Input via 8255 Port 2C
System On/Off	External Interrupt (connected to 8259 IR0)	Toggle ON_FLAG
Refill Duration	Software-timed 5.0 seconds (nominal)	Loop delay in Assembly code

Figure 1. System Specifications and Features

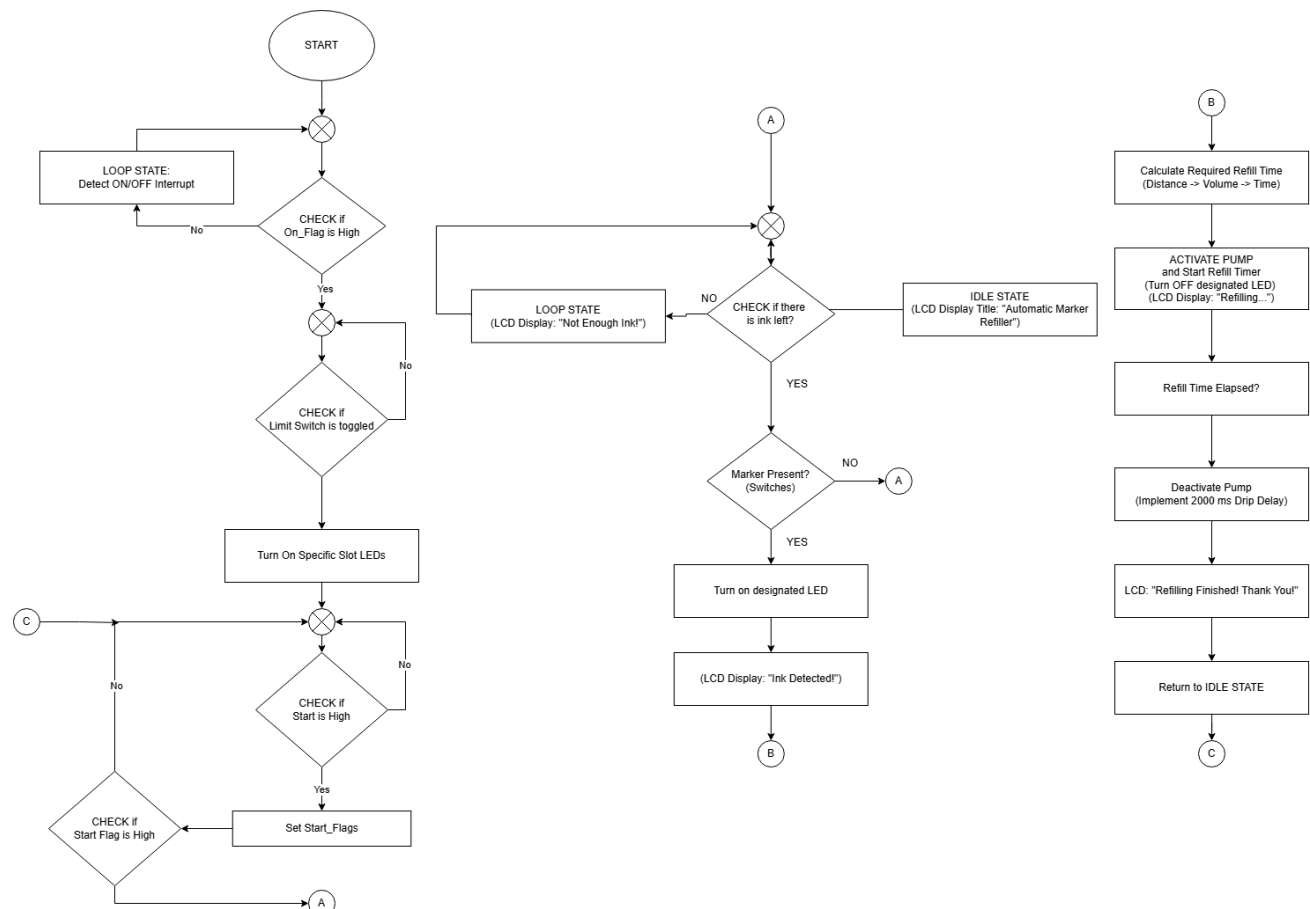


Figure 2. System Flowchart

Key Functionalities:

- Integrated Ink Level Monitoring: The system continuously monitors the main ink reservoir using a conductive sensor coupled with an ADC (Analog-to-Digital Converter). It halts all operations if the primary ink supply is critically low, displaying an error message (**NOT ENOUGH INK!**).
- Automatic Marker Detection: Four dedicated slots are monitored by switches. The system checks which slots are occupied and activates only the corresponding pumps.
- Time-Based Refilling: Refilling is not sensor-driven but time-based. Once a marker is detected, the designated pump(s) are activated for a fixed duration (5 seconds) to ensure sufficient replenishment.
- Interrupt-Driven Power Control: A single external interrupt switch allows the user to safely turn the system ON and, if necessary, shut down all active processes and return the I/O to a safe, default state.

The system operates in three distinct phases: IDLE, CHECK, and REFILL. The user initiates and interacts with the system primarily via the interrupt button and the marker slots.

Phase I: Initialization and Idle State

1. System Power On: Press the external START/INIT button (interrupt switch). The system initializes all peripherals and sets the ON_FLAG to 1.
2. IDLE State: The system enters the IDLE STATE, displaying the title message on the LCD (**AUTOMATIC MARKER REFILLER**). It remains in this state until the ON_FLAG is 0 (shutdown) or the conditions for refilling are met.

Phase II: Ink Check and Marker Detection

1. Ink Level Check: The system moves to the CHECK STATE, reading the ADC value from the ink reservoir sensor.
 - NO Ink Left (Fail): If the ink level is below the pre-set threshold (< 1), the system enters the LOOP STATE, displaying the error message (**NOT ENOUGH INK!**) and returning to the check state until the ink is replenished.
 - YES Ink Left (Pass): If ink is available, the system proceeds to check the marker slots.
2. Marker Insertion: The user inserts a marker into any of the four slots. The corresponding marker pressed switch is activated.
3. Marker Indication: The system detects the active switch(es) and turns on the designated LED for the slot, displaying the confirmation message (**INK DETECTED!**) on the LCD. The process then transitions to the refill phase.

Phase III: Refilling Process

1. Refill Preparation: The system first theoretically calculates the required refill time (Currently fixed at 5 seconds by the software).
2. Pump Activation: The system activates the pump for the detected marker slot(s) and starts the refill timer. The LCD displays the status message (**REFILLING...**).
3. Refill Timer Elapsed: The pumps remain active until the 5-second refill time elapsed is met.
4. Deactivation and Finish: The pump(s) are deactivated, and the system implements a 2000ms (2 second) drip delay to allow residual ink to settle. The LCD displays the success messages (**REFILLING FINISHED! THANK YOU!**).

5. Return to IDLE: The system then Returns to IDLE STATE to await the next marker insertion or a shutdown command.

III. Hardware Implementation

The hardware system of this project was implemented using a classic 8086 microprocessor architecture, interfacing with a suite of peripheral devices to manage the sensing, actuation, and user interface functions. The core design focuses on expanding the microprocessor's I/O capabilities and managing complex analog signals using dedicated peripheral chips.

1. Computer System and Core Components

A. Microprocessor (MPU)

- Selected System: Intel 8086MPU (U1).
- Role: The 8086 serves as the central processing unit, executing the Assembly code and generating all control signals. Its 16-bit architecture and segmented memory management govern the entire system operation.

B. Address Decoding and Data Buffering

- Address Latch (U3, U4): 74HC573 ICs are used to latch the lower 16-bit address lines (AD 0-15) when the ALE (Address Latch Enable) signal from the 8086 is asserted. This separates the multiplexed address/data bus, providing stable address lines (A 0-15).
- Data Buffer (U11, U12): 74LS245 bidirectional buffers are used to safely isolate and manage data flow (D 0-15) between the MPU and the peripheral chips, activated by the DEN (Data Enable) signal.
- Address Decoder (U2): A 74HC138 (3-to-8 decoder) is used to generate the Chip Select (CS) signals for the primary peripherals (8255s, 8259, etc.).

C. Peripheral Devices

- Programmable Interrupt Controller (PIC): An 8259 (U10) is included to manage the external START/INIT switch, which is configured as the IR0 interrupt source for the 8086.
- Programmable Peripheral Interfaces (PPI): Two 8255 ICs (U5, U9) are the main I/O expansion chips, configured in Mode\ 0 (Simple I/O) to map digital signals:
 - 8255A (U5): Interfaces with the ADC and LCD.
 - 8255B (U9): Interfaces with the Pumps and Switches.
- Analog-to-Digital Converter (ADC): An ADC0808 (U6) is used to read the analog voltage from the ink level sensor.

2. I/O Mapping and Signal Definition

The system interacts with the environment through a mix of analog and digital signals, mapped to the PPI ports as defined below.

Peripheral Component	PPI Chip	PPI Port	I/O Signals	Type
Pumps/Actuators	8255B (U9)	Port A (PA0–3)	PUMP1 to PUMP4	Digital Output
LCD Data	8255A (U5)	Port A (PA0–7)	D0 to D7	Digital Output

LCD Control	8255A (U5)	Port B (PB6,7)	RS,E (Register Select, Enable)	Digital Output
ADC Control	8255A (U5)	Port B (PB0-5)	ADDA,ADDB,ADDC,ALE,START,OE	Digital Output
ADC Data	8255A (U5)	Port C (PC0-7)	OUT1 to OUT8 (8-bit data)	Digital Input
Marker Switches	8255B (U9)	Port C (PC0-3)	MARKER1 to MARKER4	Digital Input
Start/Stop Switch	8255B (U9)	Port C (PC4)	START button	Digital Input
Ink Level Sensor	ADC0808 (U6)	IN0	Voltage signal from RV1	Analog Input

Figure 3. System Signals Table

3. Input and Output Signal Definitions

A. Digital Inputs

- Marker Detection (PC0-3): Four switches connected to Port 2C detect the mechanical presence of a marker in each slot.
- Start/Stop (PC4): A momentary push-button that signals the program to initiate the refilling sequence.
- ADC Data (PC0-7): The 8-bit digital output from the ADC0808, representing the sampled analog ink level voltage.

B. Digital Outputs

- Pump Activation (PA0-3): Four independent control lines used to drive the transistors that switch the power to the DC refill pumps (M1 to M4).
- LCD Control (PB6, 7): Dedicated lines for sending commands or data to the LCD module.
- ADC Control (PB0-5): Signals required to manage the ADC conversion cycle (channel selection, address latch, start conversion, and output enable).

C. Analog Inputs

- Ink Level Sensor (RV1): A variable resistor provides a continuous Analog Voltage signal to the IN0 channel of the ADC0808, where the voltage level is inversely proportional to the resistance (and thus proportional to the amount of ink detected).

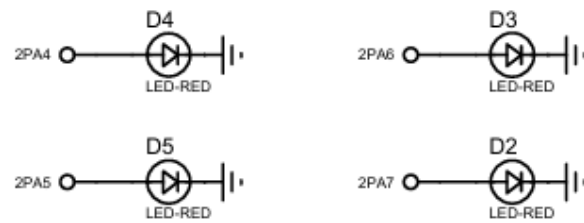
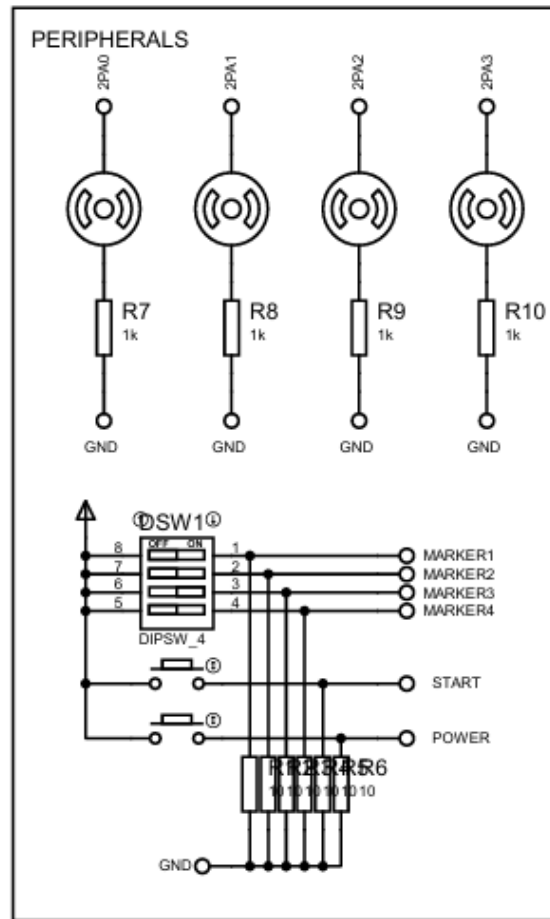


Figure 4. System Motors, Switch and Button Inputs, LED Indicators

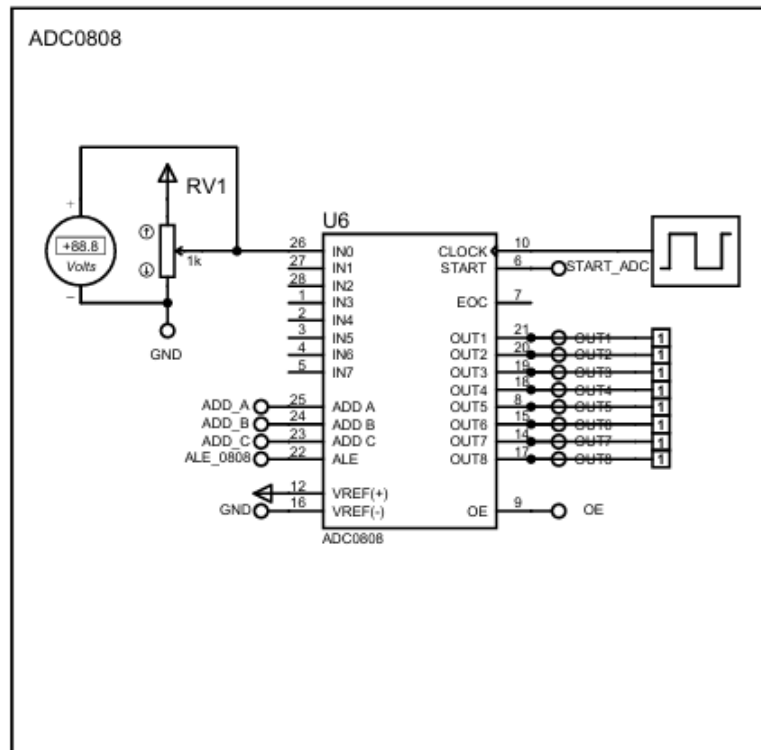


Figure 5. ADC0808

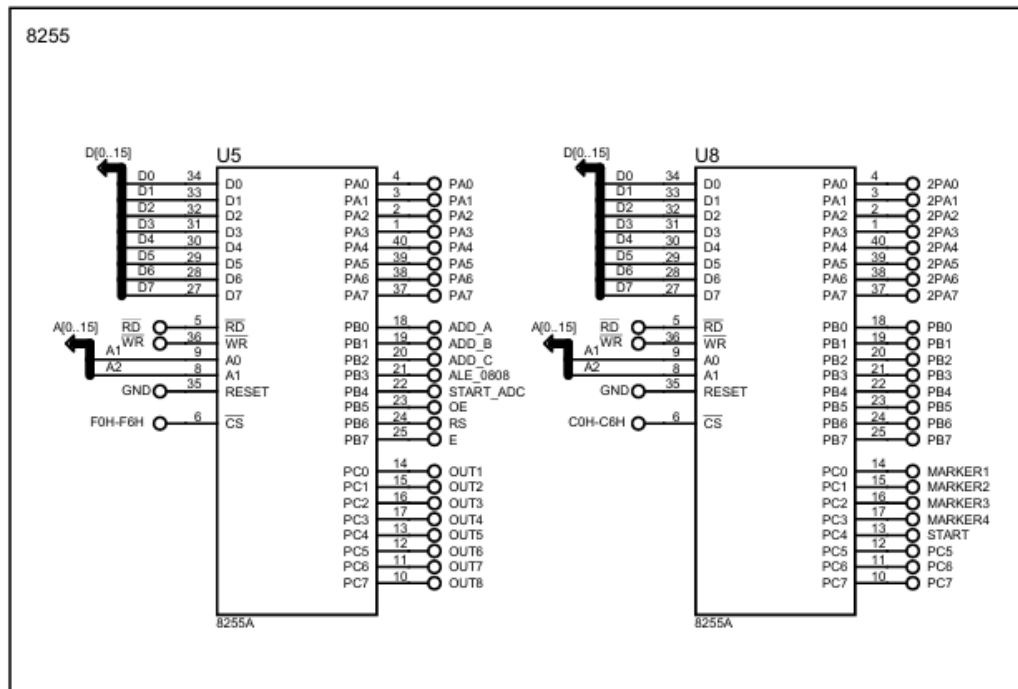
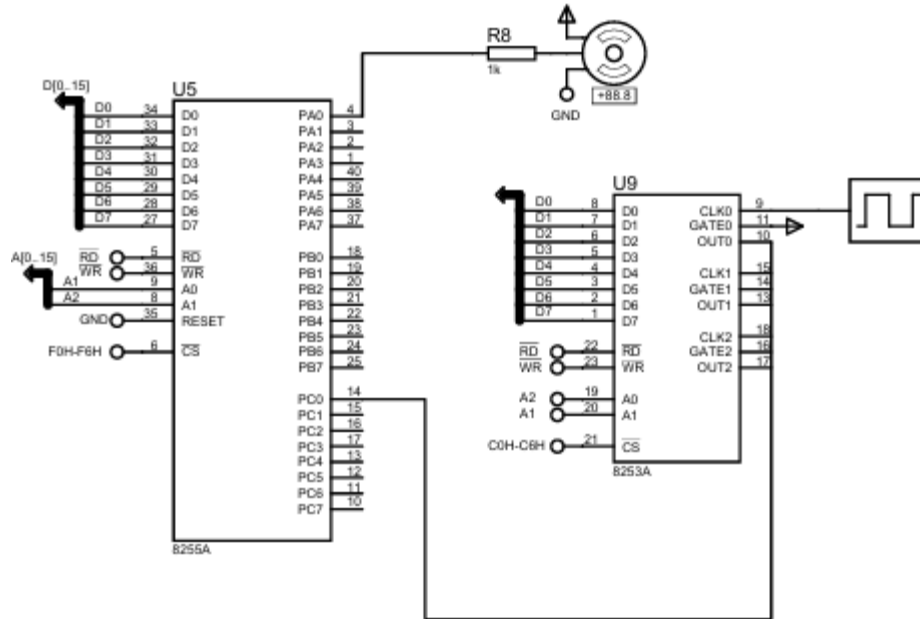


Figure 6. Setup of the 2 8255 PPIs

8255



Omission of Servo Mechanism in Final Design

During prototyping, a servo-based ink-dispensing mechanism was initially considered. However, this approach was removed due to several hardware-level limitations that prevented reliable and repeatable control when interfaced with the 8086 microprocessor.

PWM Requirements vs. 8086 Capabilities

Standard hobby servos require **precise PWM pulses**:

- High pulse: **1.0–2.0 ms** (typically 1.25 ms for mid-range)
- Low pulse: **18–19 ms**, producing a **20 ms period**

The 8086 alone cannot generate hardware-timed PWM with microsecond accuracy because:

- It has **no native PWM peripherals**
- Timing loops in software depend on instruction cycle execution, which is affected by interrupts and code branching
- Even slight timing jitter causes the servo to misinterpret the control signal

As a result, software-generated PWM was unstable and produced unpredictable servo angles.

Limitations of the 8253 Programmable Interval Timer

To solve this, an 8253 PIT was tested for hardware-timed pulse generation. The 8253 is capable of generating accurate square waves and pulse trains; however:

- Creating the correct **non-50% duty cycle** (1.25 ms high, 18.5 ms low) required combining channels or reloading values mid-cycle.

- The 8253 outputs remain **square-wave structured** unless heavily modified, making it difficult to produce servo-specific pulse widths.
- While the high and low durations were generated, the servo **did not respond** and defaulted to **its neutral 90° position**, indicating that the waveform still did not meet the servo's required signal profile (timing jitter, edge alignment, or pulse integrity issues).

Thus, even with the PIT's precision, the servo output was not usable for consistent angular control.

Alternative Consideration: 555 Timer-Based PWM

A 555 timer in astable mode was considered as another PWM source. However, several electrical factors made this impractical for a precision-controlled system:

- **Component Tolerance Issues:**
The pulse width depends on resistors and capacitors with typical tolerances of $\pm 5\%$ to $\pm 20\%$. Even small variations cause significant drift in the intended 1–2 ms pulse.
- **Temperature Drift:**
The 555's timing capacitor and internal transistor thresholds vary with temperature, causing the PWM output to shift unpredictably over time.
- **Supply Voltage Sensitivity:**
Variations in the 5V supply directly influence the output duty cycle. Minor fluctuations cause the servo to "twitch" or lose alignment.
- **Lack of Dynamic Control:**
Changing pulse width requires physically adjusting resistors or using digital potentiometers. The system needs **programmable** pulse width changes, which the 555 does not inherently support.

Because the 555 timer cannot maintain the stable $\pm 10 \mu\text{s}$ pulse resolution required by hobby servos, its output is not reliable enough for repeatable mechanical positioning.

Final Decision

Given these limitations, software jitter, PIT duty-cycle constraints, and 555 timer instability, the team decided to omit the servo mechanism entirely. A simpler and more reliable motor-driven refilling system was adopted instead, eliminating the need for precision angle control.

This change resulted in a more stable, safer, and more predictable refilling mechanism, better suited to the constraints of an 8086-based design.

IV. Software Implementation

The software implementation for the automated marker refilling machine integrates sensor detection, motor actuation, LCD messaging, interrupt handling, and analog monitoring into a coordinated control flow. Built around the **8086 microprocessor**, the system uses two **8255 PPIs**, an **8259 PIC**, and an **ADC0808** to execute automated refilling procedures based on user actions and slot occupancy. The program handles foreground refilling routines while relying on interrupt-driven events to manage slot detection and system switching.

Use Case Description

The system allows two main interactions:

1. **Initiating the refilling process**
2. **Monitoring slot occupation and ink-level conditions**

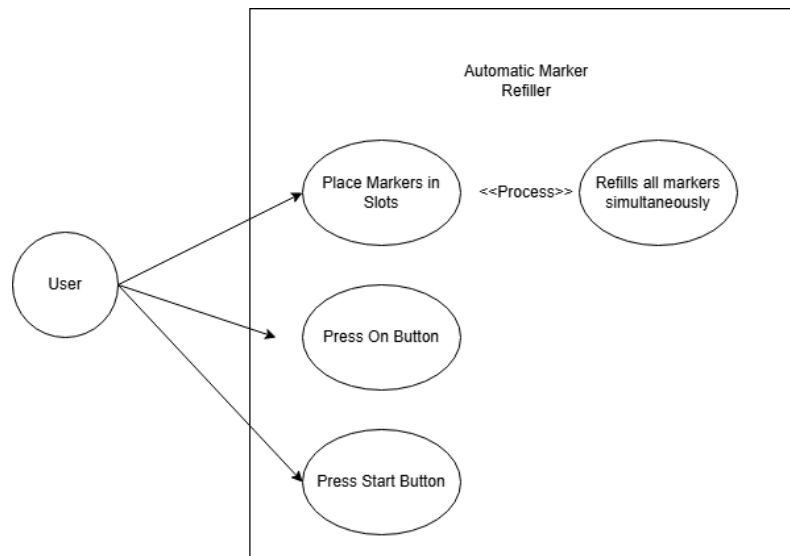


Figure 7. System Interaction Diagram

Once the system is powered, the user may begin the refill function through a button press. The software first checks which of the four marker slots contain physical markers. Slot detection is performed through digital inputs connected to PORT C of the 8255. Only the slots containing markers are scheduled for refilling.

When a refill command is issued, the system performs the following:

- Identifies occupied slots
- Reads analog ink-level data via the ADC0808

- Drives the corresponding DC motors
- Displays system messages (e.g., “REFILLING SLOT 1”, “COMPLETE”, “NO MARKER DETECTED”) on the LCD

Foreground operations handle the timed motor actuation. Interrupt-driven routines update system flags, detect slot changes, and monitor emergency stop conditions.

This model emphasizes the flow of interaction, showing how the user’s single action is translated into automated slot detection, ink-level validation, and multi-motor control.

Programmable ICs and Port Mapping

8255 PPI - 1 INTERFACE

Function	Port	Address
LCD Data (Menu/Status)	PORTA	F0H
ADC0808 Control/Initialization Bits	PORTB	F2H
ADC0808 Input Bit Steps	PORTC	F4H
Command Register		F6H

8255 PPI - 2 INTERFACE (Sensors and Motors)

Function	Port	Address
Motor Outputs (4 DC motors)	PORTA	C0H
Indicators / Signal LEDs	PORTA	C0H
Marker Slot Sensors	PORTC	C4H
Start Button	PORTC	C4H
Command Register	N/A	C6H

The port addresses used in the system are assigned within two primary ranges: **F0H–F6H** for the first 8255 PPI, which manages the LCD and the ADC signal interface, and **C0H–C6H** for the second 8255 PPI, which handles motor outputs and switch inputs,. These address allocations ensure that each peripheral device is uniquely selected through the decoder circuitry, allowing the microprocessor to reliably access all input and output components without conflict.

Foreground and Interrupt Routines

The refilling process is executed in the foreground.

Interrupt-driven routines handle:

- ON/OFF switch
- Clearing of Flags and Output Ports
- Emergency shutdown requests

Flags updated by the ISR determine whether the main routine must initiate a forced shutdown or enter standby.

Logical Variables and System Flags

The system relies on several flag variables that must be continuously monitored in order to coordinate user actions, slot detection, and motor activation. These flags act as internal status indicators that guide the foreground routine during the refilling process.

The following logical variables are implemented:

- **ON_FLAG** – indicates whether the system has been powered on and is ready for operation.
- **START_FLAG** – set when the user initiates the refilling sequence. This flag triggers the program to begin evaluating slot conditions and activating motors.
- **MRK1_FLAG – MRK4_FLAG** – four slot-specific flags that identify whether a marker is physically inserted into each of the four refill slots. Only the slots with their corresponding flag set are included in the refill cycle.

These flags maintain synchronization between the main loop and all real-time inputs. By updating slot states, detecting start conditions, and enabling safe activation of motors, the system ensures consistent and controlled ink dispensing while preventing unintended or unsafe operations.

The system uses two 8255 PPIs to manage all input and output operations required for ink-refilling automation. Each PPI is assigned a specific functional group to simplify decoding and ensure reliable data flow between the microprocessor, ADC, sensors, motors, and the display interface.

PPI 1 – LCD and ADC Interface (Addresses F0H–F6H)

PPI 1 handles all display functions and ADC interfacing. The ports are assigned as follows:

Port A (PA0–PA7) – LCD Data Bus

Port A is configured as an output port and provides the 8-bit data bus for the LCD module (D0–D7).

This port is responsible for sending character codes, commands, and configuration data to the display.

Port B (PB0–PB7) – ADC Control Signals

Port B is also configured as an output port and drives the control lines of the ADC0808. These control bits include:

- **ADD-A, ADD-B, ADD-C** (channel selection)
- **START-ADC** (initiate conversion)
- **OE** and other control strobes depending on design

This allows the microprocessor to select the active analog channel and trigger conversions.

Port C (PC0–PC7) – ADC Digital Output

Port C is configured as an input port and receives the **8-bit digital result** from the ADC0808 following each conversion cycle.

This value represents the measured ink level of the currently selected refill slot.

PPI 2 – Motor Drivers, Indicator LEDs, and Switch Inputs (Addresses C0H–C6H)

PPI 2 is responsible for handling all physical marker detection and motor operations.

Port A (PA0–PA7) – Motor Control and Slot Indicator Lamps

Port A is configured as an output port and is used to drive:

- The DC motors responsible for ink pumping
- Indicator LEDs that correspond to each marker slot

Each bit is mapped to a specific motor or status output.

Port B (PB0–PB7) – Unused

Port B is not used in the current prototype configuration but remains available for future expansion (e.g., additional sensors, actuators, or communication lines).

Port C (PC0–PC7) – Input Signals (Limit Switches and Start Button)

Port C is configured as an input port and interfaces with real-time hardware signals:

- **Marker limit switches (MARKER1–MARKER4)** used for slot detection
- **START** pushbutton to initiate the refilling sequence
- Additional input pins reserved for safety or extension

These signals determine which slots are active and when the system should begin operation.

Motor Control Logic

Each motor corresponds to one refill slot.

Only slots where switches indicate marker presence receive activation.

A logic “1” on **PORT A** activates the motor:

Slot	Motor Bit	Meaning
Slot 1	Bit 0	Motor On
Slot 2	Bit 1	Motor On
Slot 3	Bit 2	Motor On
Slot 4	Bit 3	Motor On

The program concatenates all required bits to form a single output byte written to **PORT A, 2nd 8255**.

LCD Output Messages

- The system uses predefined LCD messages to give the user clear, step-by-step feedback during operation. At startup, “**AUTOMATIC MARKER**” and “**REFILLER**” identify the machine and indicate readiness. When the user begins the process, “**REFILLING...**” appears, followed by either “**INK DETECTED!**” or “**NOT ENOUGH INK!**” depending on the ADC ink-level reading.
- After a successful cycle, the display shows “**REFILLING FINISHED!**” and “**THANK YOU!**”, confirming that the marker can be removed. During shutdown, the LCD outputs “**SHUTTING DOWN...**” to indicate that all operations are stopping. These messages ensure clear, continuous communication between the system and the user.

V. References

daniel.abonyi@marketingastro.com. (2025, February 20). The Science Behind ink:

Composition, Chemistry, and Applications - Zebra Pen EU. *Zebra Pen EU*.

<https://zebrapeneu.com/the-science-behind-ink-composition-chemistry-and-applications/>

Brazeau, L., & Gaudreau, M. (2006). Ballpoint Pen Inks: The Quantitative Analysis of ink solvents on Paper by Solid-Phase Microextraction. *Journal of Forensic Sciences*, 52(1), 209–215. <https://doi.org/10.1111/j.1556-4029.2006.00299.x>

Control Systems Engineering. (n.d.). Google Books.

https://books.google.com.ph/books/about/Control_Systems_Engineering.html?id=fj6MEAA_AQBAJ&redir_esc=y

CERTIFICATION

We attest and certify that the implementation of this project is authentic, no portion/component/module is AI-generated, copied/generated from unscrupulous sources, has no copyright/patent violations, nor violates any legal, ethical or moral standards of the university.

Signed this 28th day of November, 2025 at Cebu City, Philippines.

Attested by:

Ephraim Joshua M. Flores

Project Leader

Alexa Gomez

Member

Riley Sy

Member

<IMPORTANT NOTE: Delete this part before printing. Print the certification page with your manually filled full name and fresh signatures affixed to it then scan/picture it before converting it to PDF and uploading it to Canvas